

- [Getting Started with cuML's accelerator mode](#)

Using Accelerated Hardware

- [Train a CNN to classify handwritten digits on the MNIST dataset using Flax NNX API](#)
- [Train a Vision Transformer \(ViT\) for image classification with JAX](#)
- [Text classification with a transformer language model using JAX](#)

Featured examples

- [Train a miniGPT language model with JAX AI Stack](#)
- [LoRA/QLoRA finetuning for LLM using Tunix](#)
- [Parameter-efficient fine-tuning of Gemma with LoRA and QLoRA](#)
- [Loading Hugging Face Transformers Checkpoints](#)
- [8-bit Integer Quantization in Keras](#)
- [Float8 training and inference with a simple Transformer model](#)
- [Pretraining a Transformer from scratch with KerasHub](#)
- [Simple MNIST convnet](#)
- [Image classification from scratch using Keras 3](#)
- [Image Classification with KerasHub](#)

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from scipy import stats
```

```
df=pd.read_csv('walmart_data.txt')
print(df.head())
```

```

  User_ID Product_ID Gender  Age  Occupation City_Category  \
0  1000001  P00069042    F  0-17          10             A
1  1000001  P00248942    F  0-17          10             A
2  1000001  P00087842    F  0-17          10             A
3  1000001  P00085442    F  0-17          10             A
4  1000002  P00285442    M  55+          16             C

  Stay_In_Current_City_Years  Marital_Status  Product_Category  Purchase
0                             2                0                 3       8370
1                             2                0                 1      15200
2                             2                0                12       1422
3                             2                0                12       1057
4                             4+                0                 8       7969
```

```
print(df.shape)
```

```
(550068, 10)
```

```
print(df.info())
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 550068 entries, 0 to 550067
Data columns (total 10 columns):
#   Column                                Non-Null Count  Dtype
---  -
0   User_ID                              550068 non-null int64
1   Product_ID                           550068 non-null object
2   Gender                               550068 non-null object
3   Age                                   550068 non-null object
4   Occupation                            550068 non-null int64
5   City_Category                         550068 non-null object
6   Stay_In_Current_City_Years            550068 non-null object
7   Marital_Status                        550068 non-null int64
8   Product_Category                      550068 non-null int64
9   Purchase                              550068 non-null int64
dtypes: int64(5), object(5)
memory usage: 42.0+ MB
None
```

```
print(df.describe())
```

```

count      User_ID      Occupation  Marital_Status  Product_Category  \
count  5.500680e+05  550068.000000  550068.000000  550068.000000
mean    1.003029e+06    8.076707    0.409653    5.404270
std     1.727592e+03    6.522660    0.491770    3.936211
min     1.000001e+06    0.000000    0.000000    1.000000
25%     1.001516e+06    2.000000    0.000000    1.000000
```

```
50%    1.003077e+06    7.000000    0.000000    5.000000
75%    1.004478e+06   14.000000    1.000000    8.000000
max     1.006040e+06   20.000000    1.000000   20.000000
```

```

      Purchase
count  550068.000000
mean    9263.968713
std     5023.065394
min      12.000000
25%     5823.000000
50%     8047.000000
75%    12054.000000
max    23961.000000
```

```
# changing datatype int64 to object
columns=['Occupation','Marital_Status','Product_Category']
df[columns]=df[columns].astype('object')
df.dtypes
```

0	
User_ID	int64
Product_ID	object
Gender	object
Age	object
Occupation	object
City_Category	object
Stay_In_Current_City_Years	object
Marital_Status	object
Product_Category	object
Purchase	int64

dtype: object

```
df.describe(include="all")
```

	User_ID	Product_ID	Gender	Age	Occupation	City_Category	Stay_In_Current_City_Years	Marital_Status	Product_Category
count	5.500680e+05	550068	550068	550068	550068.0	550068	550068	550068.0	550068.0
unique	NaN	3631	2	7	21.0	3	5	2.0	2.0
top	NaN	P00265242	M	26-35	4.0	B	1	0.0	0.0
freq	NaN	1880	414259	219587	72308.0	231173	193821	324731.0	324731.0
mean	1.003029e+06	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN
std	1.727592e+03	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN
min	1.000001e+06	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN
25%	1.001516e+06	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN
50%	1.003077e+06	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN
75%	1.004478e+06	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN
max	1.006040e+06	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN

```
# finding null values
df.isna().sum()
```

	0
User_ID	0
Product_ID	0
Gender	0
Age	0
Occupation	0
City_Category	0
Stay_In_Current_City_Years	0
Marital_Status	0
Product_Category	0
Purchase	0

dtype: int64

```
gender_counts=df['Gender'].value_counts()
print(gender_counts)
```

```
Gender
M    414259
F    135809
Name: count, dtype: int64
```

```
age_counts=df['Age'].value_counts()
print(age_counts)
```

```
Age
26-35    219587
36-45    110013
18-25     99660
46-50     45701
51-55     38501
55+       21504
0-17      15102
Name: count, dtype: int64
```

```
Stay_In_Current_City_Years_counts=df['Stay_In_Current_City_Years'].value_counts()
print(Stay_In_Current_City_Years_counts)
```

```
Stay_In_Current_City_Years
1     193821
2     101838
3       95285
4+     84726
0       74398
Name: count, dtype: int64
```

```
marital_status_counts=df['Marital_Status'].value_counts()
print(marital_status_counts)
```

```
Marital_Status
0     324731
1     225337
Name: count, dtype: int64
```

#unique attributes

```
unique_category_count=df['Product_Category'].nunique()
print(unique_category_count)
```

20

```
unique_city_category_count=df['City_Category'].nunique()
print(unique_city_category_count)
```

3

```
unique_Product_Id_count=df['Product_ID'].nunique()
print(unique_Product_Id_count)
```

3631

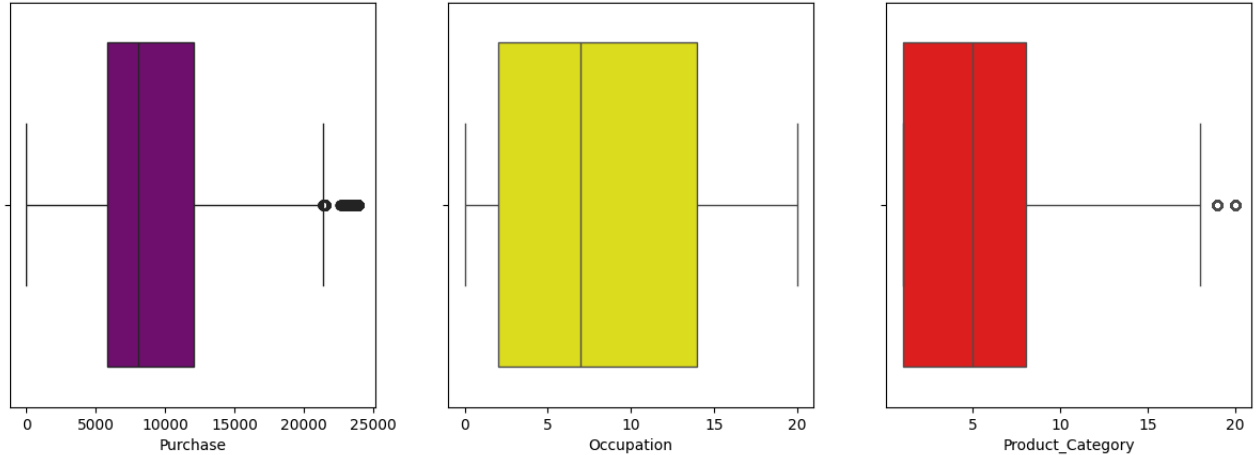
```
unique_User_Id_count=df['User_ID'].nunique()
print(unique User Id count)
```

5891

Outliers detection using BoxPlots

```
fig,axis=plt.subplots(nrows=1,ncols=3,figsize=(15,2))
fig.subplots_adjust(top=2)
sns.boxplot(data=df,x='Purchase',ax=axis[0],color='purple')
sns.boxplot(data=df,x='Occupation',ax=axis[1],color='yellow')
sns.boxplot(data=df,x='Product_Category',ax=axis[2],color='red')
```

<Axes: xlabel='Product_Category'>



Mean vs Median comparison

```
mean_val=df['Purchase'].mean()
median_val=df['Purchase'].median()
print('Mean:',mean_val)
print('Median:',median_val)
```

```
Mean: 9263.968712959126
Median: 8047.0
```

Gender Based Mean Analysis

```
avg_by_gender=df.groupby('Gender')['Purchase'].mean()
print(avg_by_gender)
```

```
Gender
F    8734.565765
M    9437.526040
Name: Purchase, dtype: float64
```

Sampling and CLT Visualisation

```
def confidence_interval(data,confidence=0.95):
    sample_mean=np.mean(data)
    sample_std=np.std(data)
    n=len(data)
    z=1.96
    margin_error=z*(sample_std/np.sqrt(n))
    return sample_mean-margin_error,sample_mean+margin_error
```

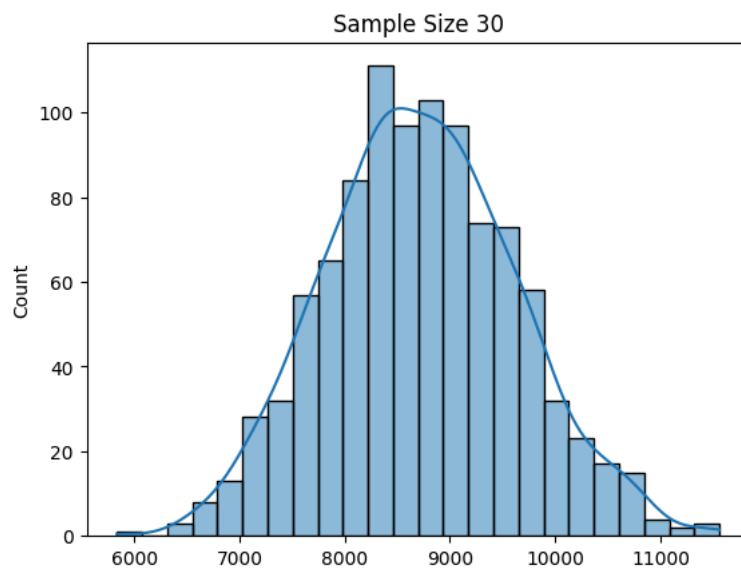
MALE & FEMALE Samples

```
male_data=df[df['Gender']=='M']['Purchase']
female_data=df[df['Gender']=='F']['Purchase']
male_ci=confidence_interval(male_data)
female_ci=confidence_interval(female_data)
print('Male CI:',male_ci)
print('Female CI:', female_ci)
```

```
Male CI: (np.float64(9422.019181136464), np.float64(9453.032899808066))
Female CI: (np.float64(8709.21117458943), np.float64(8759.92035572152))
```

CLT Visualisation

```
def sample_means(data,sample_size,iteration=1000):
    means=[]
    for _ in range(iteration):
        sample=np.random.choice(data,sample_size)
        means.append(np.mean(sample))
    return means
means_30=sample_means(female_data,30)
means_100=sample_means(female_data,100)
plt.figure()
sns.histplot(means_30,kde=True)
plt.title('Sample Size 30')
plt.show()
plt.figure
sns.histplot(means_100,kde=True)
plt.title('Sample Size 100')
plt.show
```

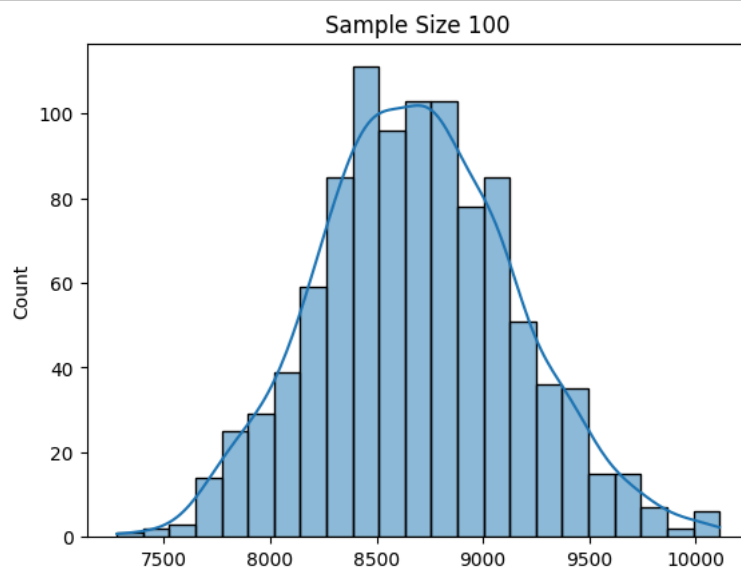


`matplotlib.pyplot.show`
 def show(*args, **kwargs) -> None

</usr/local/lib/python3.12/dist-packages/matplotlib/pyplot.py>
 Display all open figures.

Parameters

 block : bool, optional



Married VS Unmarried

```
martial_avg=df.groupby('Marital_Status')['Purchase'].mean()
print(martial_avg)
```

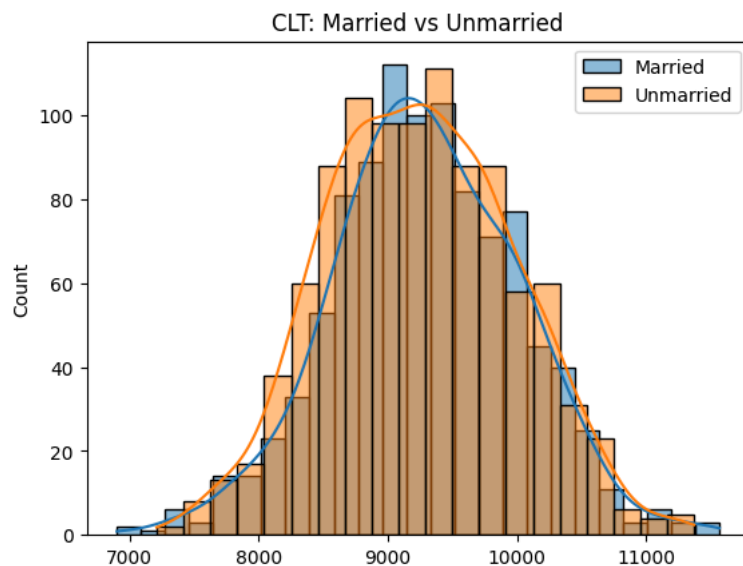
```
Marital_Status
0    9265.907619
1    9261.174574
Name: Purchase, dtype: float64
```

```
married=df[df['Marital_Status']==1]['Purchase']
unmarried=df[df['Marital_Status']==0]['Purchase']
```

```
def sample_means(data,sample_size,iterations=1000):
    means=[]
    for _ in range(iterations):
        sample=np.random.choice(data,sample_size)
        means.append(np.mean(sample))
    return means
```

```
married_means=sample_means(married,50)
unmarried_means=sample_means(unmarried,50)
```

```
plt.figure()
sns.histplot(married_means,kde=True,label='Married')
sns.histplot(unmarried_means,kde=True,label='Unmarried')
plt.legend()
plt.title('CLT: Married vs Unmarried')
plt.show()
```



```
# Comparing Confidence Interval
```

```
def confidence_interval(data):
    mean=np.mean(data)
    std=np.std(data)
    n=len(data)
    z=1.96
    margin=z*(std/np.sqrt(n))
    return mean-margin,mean+margin
print("Married CI:",confidence_interval(married))
print("Unmarried CI:",confidence_interval(unmarried))
```

```
Married CI: (np.float64(9240.460092386264), np.float64(9281.889055778483))
Unmarried CI: (np.float64(9248.616127075364), np.float64(9283.19911076765))
```

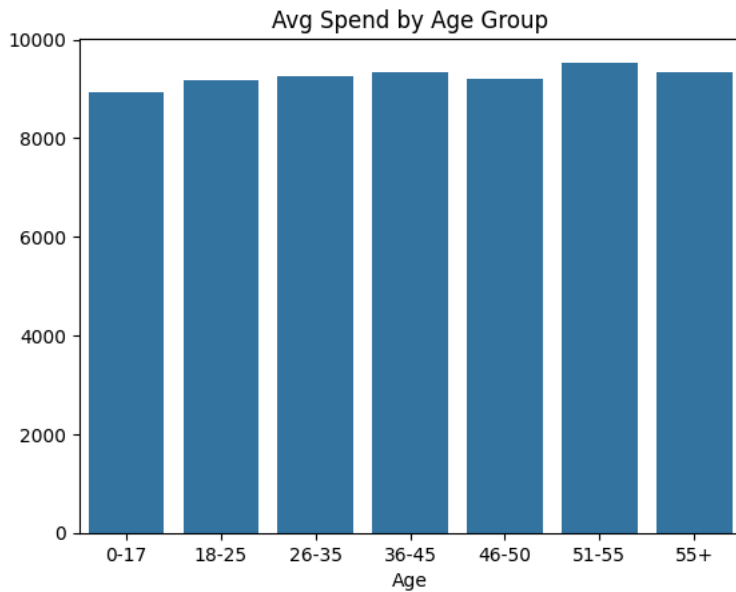
```
# Age Group Analysis
```

```
bins=[0,17,25,35,50,100]
labels=['0-17','18-25','26-35','36-50','51+']
age_avg=df.groupby('Age')['Purchase'].mean()
print(age_avg)
```

```
Age
0-17    8933.464640
18-25    9169.663606
26-35    9252.690633
36-45    9331.350695
```

```
46-50    9208.625697
51-55    9534.808031
55+      9336.280459
Name: Purchase, dtype: float64
```

```
plt.figure()
sns.barplot(x=age_avg.index,y=age_avg.values)
plt.title('Avg Spend by Age Group')
plt.show()
```



Gender Visualisation

```
age_gender_avg=df.groupby(['Age', 'Gender'])['Purchase'].mean().reset_index()
print(age_gender_avg)
```

	Age	Gender	Purchase
0	0-17	F	8338.771985
1	0-17	M	9235.173670
2	18-25	F	8343.180201
3	18-25	M	9440.942971
4	26-35	F	8728.251754
5	26-35	M	9410.337578
6	36-45	F	8959.844056
7	36-45	M	9453.193643
8	46-50	F	8842.098947
9	46-50	M	9357.471509
10	51-55	F	9042.449666
11	51-55	M	9705.094802
12	55+	F	9007.036199
13	55+	M	9438.195603

INSIGHT

1. Gender spending behaviour- males generally spend slightly more transaction than females
 2. Confidence Interval insight- if male and female cls overlap slightly difference exist but not very strong, if they don't overlap strongly, statistically significant difference
 3. married customers often spend more
 4. high spending groups; 26-35, 36-50
 5. low spending groups- 0-17, 51+
 6. CLT observation- with larger samples distribution becomes normal, result becomes more reliable

✓ RECOMMENDATION

1. target high value segments- age-26-50, male customers, married customers
 2. create gender based strategies
 3. family oriented marketing
 4. age based targeting
 5. location strategy - if big city spends more focus inventory there, if smaller cities growing expand aggressively
 6. pricing strategy - if certain group spend more introduce premium pricing, if spending is similar keep uniform pricing
 7. product recommendations- use behaviour pattern

```
plt.figure()
sns.barplot(x='Age', y='Purchase', hue='Gender', data=age_gender_avg)
plt.title('Avg-wise Male vs Female')
plt.show()
```

